

P NEEHARIKA-2520090233

SKILL 6 (SESSION 11, SESSION 12)

Task 1:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
root@HP: ~/skill_week6 task1_history.c *
GNU nano 7.2
#include <stdio.h>
#include <string.h>

#define MAX 5
#define SIZE 100

int main() {
    char history[MAX][SIZE];
    char command[SIZE];
    int count = 0, i;

    printf("Command History Buffer\n");
    printf("Enter commands (type exit to stop)\n");

    while (1) {
        printf("Command: ");

        if (fgets(command, SIZE, stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strlen(command) == 0)
            continue;

        if (count < MAX) {
            strcpy(history[count], command);
            count++;
        } else {
            for (i = 0; i < MAX - 1; i++)
                strcpy(history[i], history[i + 1]);

            strcpy(history[MAX - 1], command);
            printf("Buffer full: oldest entry removed.\n");
        }
    }

    printf("\nStored Command History:\n");

    for (i = 0; i < count; i++)
        printf("%d. %s\n", i + 1, history[i]);

    printf("\nTotal entries: %d\n", count);
    printf("History consistency: Valid\n");

    return 0;
}
```

Terminal showing compilation, execution, stored history, and buffer-limit handling.

```
root@HP:~/skill_week6# gcc task1_history.c -o task1_history
root@HP:~/skill_week6# ./task1_history
Command History Buffer
Enter commands (type exit to stop)
Command: ls
Command: pwd
Command: data
Command: whoami
Command: gcc
Command: uname
Buffer full: oldest entry removed.
Command: exit

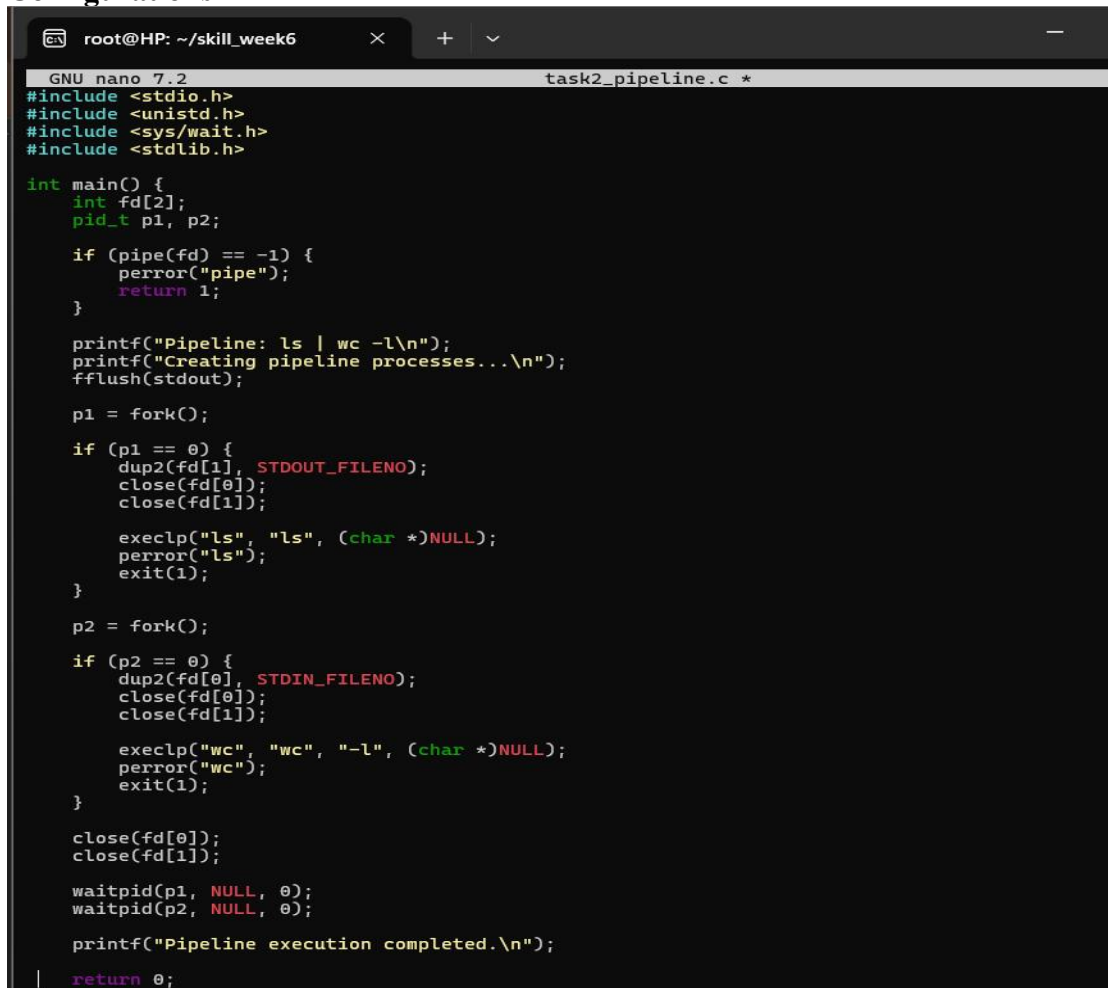
Stored Command History:
1. pwd
2. data
3. whoami
4. gcc
5. uname

Total entries: 5
History consistency: Valid
root@HP:~/skill_week6# |
```

Messages

Task 2

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations



```
GNU nano 7.2 task2_pipeline.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd[2];
    pid_t p1, p2;

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    printf("Pipeline: ls | wc -l\n");
    printf("Creating pipeline processes...\n");
    fflush(stdout);

    p1 = fork();
    if (p1 == 0) {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);

        execlp("ls", "ls", (char *)NULL);
        perror("ls");
        exit(1);
    }

    p2 = fork();
    if (p2 == 0) {
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        close(fd[1]);

        execlp("wc", "wc", "-l", (char *)NULL);
        perror("wc");
        exit(1);
    }

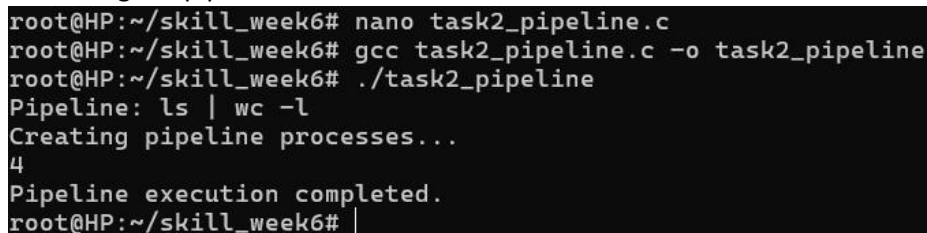
    close(fd[0]);
    close(fd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    printf("Pipeline execution completed.\n");

    return 0;
}
```

Terminal showing the pipeline execution and



```
root@HP:~/skill_week6# nano task2_pipeline.c
root@HP:~/skill_week6# gcc task2_pipeline.c -o task2_pipeline
root@HP:~/skill_week6# ./task2_pipeline
Pipeline: ls | wc -l
Creating pipeline processes...
4
Pipeline execution completed.
root@HP:~/skill_week6#
```

output.

Observation — Session 11

The command history buffer and pipeline structures were implemented and tested successfully.

SESSION 12 — SKILL WEEK 6

Task 1

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```
root@HP: ~
GNU nano 7.2 task3
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stdlib.h>

int main() {
    int fd[2];
    pid_t p1, p2;

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    printf("Pipe and Redirection Demonstration\n");
    fflush(stdout);

    p1 = fork();

    if (p1 == 0) {
        close(fd[0]);
        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);

        execlp("printf", "printf",
               "apple\nbanana\norange\n",
               (char *)NULL);
        perror("printf");
        exit(1);
    }

    p2 = fork();

    if (p2 == 0) {
        int file = open("output.txt",
                       O_WRONLY | O_CREAT | O_TRUNC,
                       0644);

        if (file == -1) {
            perror("open");
            exit(1);
        }

        close(fd[1]);
        dup2(fd[0], STDIN_FILENO);
        dup2(file, STDOUT_FILENO);

        close(fd[0]);
        close(file);

        execlp("tr", "tr", "a-z", "A-Z",
               (char *)NULL);
        perror("tr");
        exit(1);
    }

    close(fd[0]);
    close(fd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    printf("Process execution completed.\n");
    printf("Data redirected to output.txt\n");

    return 0;
}

root@HP:~# gcc task3_pipe_redirect.c -o task3_pipe_redirect
root@HP:~# ./task3_pipe_redirect
Pipe and Redirection Demonstration
Process execution completed.
Data redirected to output.txt
root@HP:~# cat output.txt
APPLE
BANANA
ORANGE
root@HP:~# APPLE
BANANA
ORANGE
```

Task 2

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd1[2], fd2[2];
    pid_t p1, p2, p3;

    if (pipe(fd1) == -1 || pipe(fd2) == -1) {
        perror("pipe");
        return 1;
    }

    printf("Pipeline: printf | tr | sort\n");
    fflush(stdout);

    p1 = fork();

    if (p1 == 0) {
        dup2(fd1[1], STDOUT_FILENO);

        close(fd1[0]);
        close(fd1[1]);
        close(fd2[0]);
        close(fd2[1]);

        execlp("printf", "printf",
              "pear\napple\nbanana\n",
              (char *)NULL);
        perror("printf");
        exit(1);
    }

    p2 = fork();

    if (p2 == 0) {
        dup2(fd1[0], STDIN_FILENO);
        dup2(fd2[1], STDOUT_FILENO);

        close(fd1[0]);
        close(fd1[1]);
        close(fd2[0]);
        close(fd2[1]);

        execlp("tr", "tr", "a-z", "A-Z",
              (char *)NULL);
        perror("tr");
        exit(1);
    }
}
```

Pipelines

```

    p3 = fork();

    if (p3 == 0) {
        dup2(fd2[0], STDIN_FILENO);

        close(fd1[0]);
        close(fd1[1]);
        close(fd2[0]);
        close(fd2[1]);

        execlp("sort", "sort", (char *)NULL);
        perror("sort");
        exit(1);
    }

    close(fd1[0]);
    close(fd1[1]);
    close(fd2[0]);
    close(fd2[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
    waitpid(p3, NULL, 0);

    printf("All pipeline processes completed.\n");

    return 0;
}

```

Terminal showing the long pipeline execution and final output.

```

root@HP: ~
root@HP:~# nano task4_multi_pipeline.c
root@HP:~# gcc task4_multi_pipeline.c -o task4_multi_pipeline
root@HP:~# ./task4_multi_pipeline
Pipeline: printf | tr | sort
APPLE
BANANA
PEAR
All pipeline processes completed.
root@HP:~#

```

Observation — Session 12

Multiple pipes and connected processes were implemented and tested successfully, demonstrating process communication, redirection, and synchronization.